

JavaScript Basic

JavaScript 기초

JavaScript 기초 다지기

JavaScript의 다양한 연산

☑ 산술연산



$+$	$-$	$*$	$/$	$\%$
덧셈	뺄셈	곱셈	나눗셈	나머지

- 자바스크립트 산술 연산자는 사칙연산과 나머지 연산을 말합니다. $+$, $-$ 기호는 덧셈과 뺄셈 연산을 진행합니다. $*$, $/$ 기호는 곱셈과 나눗셈 연산을 진행합니다. $\%$ 기호는 몫을 나눈 나머지를 결과로 반환합니다.

✓ 산술연산

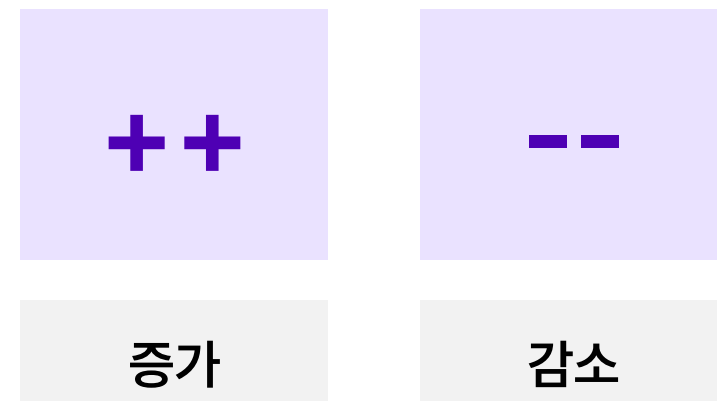


```
let a = 10
let b = 3

console.log(a+b); //13
console.log(a-b); //7
console.log(a*b); //30
console.log(a/b); //3.33333
console.log(a%b); //1
```

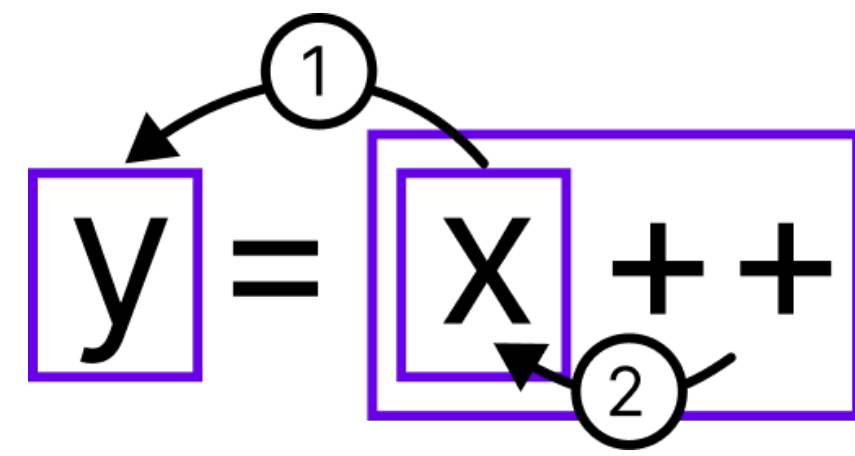
- 자바스크립트 나머지 정리는 % 기호를 사용합니다. 이 기호는 X를 Y로 나눴을 때 나오는 몫과 나머지 중에서 나머지만 출력합니다.

☑ 증감연산



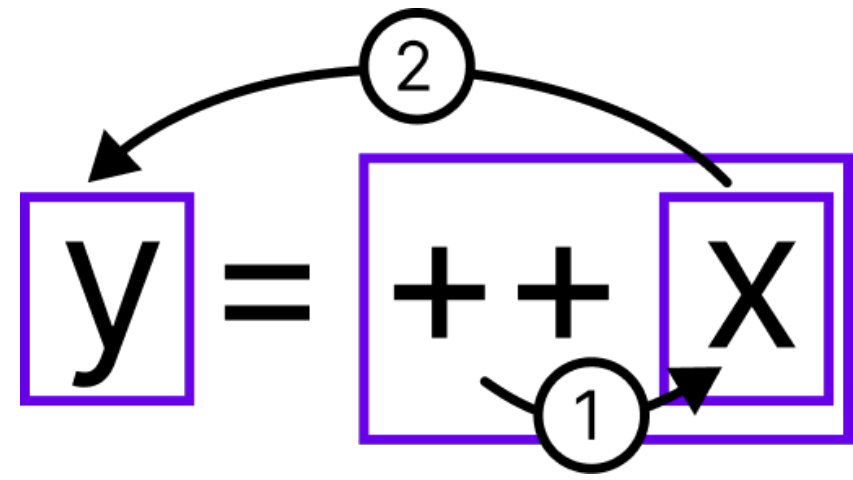
- 증감 연산자란, 변수의 값을 1씩 증가시키거나 1씩 감소시키는 연산자로,
- 늘리거나 줄인다는 뜻으로 증감이라는 이름이 붙었다고 합니다.
- 증감 연산자는 2종류가 있으며,
- 값을 1씩 증가시키는 것을 Increment 연산자,
- 값을 1씩 감소시키는 것을 Decrement 연산자라고 한다.

☑ 전위 / 후위 연산



후위연산

연산자가 변수의 뒤에 위치
후위 연산자는 변수에 먼저 할당(대입)한 뒤에 연산



전위연산

연산자가 변수의 앞에 위치
전위 연산자는 연산을 먼저 한 뒤에 변수에 할당(대입)

☑ 복합 대입연산



```
let a = 2;  
let b = 'elice';  
  
console.log(a += 3); // 5  
console.log(b += ' world'); // "elice world"
```

+=

증가대입

-=

감소대입

***=**

곱대입

- 대입 연산자는 변수에 값을 대입할 때 사용하는 이항 연산자이며,
- 피연산자들의 결합 방향은 오른쪽에서 왼쪽이다.
- 또한, 앞서 살펴본 산술 연산자와 결합한 다양한 복합 대입 연산자가 존재한다.

✔ 비교연산



변수의 값을 어떤 값 또는 다른 변수와 비교할 때 사용하는 것이 비교 연산자(Comparison Operator)입니다.

	설명		설명
a=b	A에 b를 대입한다.	a!=b	A와 B는 같지않다.
a==b	A와 b는 같다	a>b a<b	A는 b보다 크다. A는 b보다 작다.
a===b	A와 B는 data type도 같다	a>=b a<=b	A는 b보다 크거나 같다. A는 b보다 작거나 같다.

✔ 비교연산



```
const a = 1
const b = 3

console.log(a===b)//false

function isEqual(x, y) {
  return x === y
}

console.log(isEqual(1,1))//true
console.log(isEqual(2,'2'))//false
```

☑ 생각해보기



```
0 == false
```

true일까 false일까?

☑ 생각해보기



```
0 == false
```

true일까 false일까?

정답은 true!

☑ 생각해보기



==는 타입을 자동으로 맞춰줌

```
' ' == false // true  
'0' == 0 // true  
null == undefined // true
```


☑ 생각해보기



따라서 == 대신 **===** 사용 권장

==

느슨한 비교 방식으로
타입이 달라도 변환해 비교하기 때문에
예상치 못한 결과가 자주 나옵니다.

===

엄격한 비교 방식으로 타입과 값이 모두
같아야 true. 의도가 명확하고 버그
가능성이 낮습니다.

☑ 논리연산



&&

그리고

||

또는

!

부정

자바스크립트 논리 연산자는 값을 true, false로 반환합니다. || 연산자는 둘 중 하나만 true여도 전체를 true 반환합니다. && 연산자는 둘 다 true이면, 전체를 true로 반환합니다.

☑ 논리연산



```
const a = 3;
const b = 5;
const result1 = (a<b) || (a>b); // true or false --> true
const result2 = (a<b) && (a>b); // true and false --> false
const result3 = !(a<b); // false(true) --> false

console.log(result1, result2, result3);
```

- $X \parallel Y$ 논리 연산자는 수학에서 OR(또는)과 같습니다. X와 Y의 값 중에서 하나만 true이면, 전체도 true입니다.
- $X \&\& Y$ 논리 연산자는 수학에서 AND(그리고)와 같습니다. X와 Y값이 모두 true를 만족해야 전체도 true를 반환합니다.
- $!X$ 연산자는 수학의 $\sim$ (NOT)과 같습니다. X값이 true이면 false를 반환하고, X값이 false이면 true를 반환합니다.

조건문

④ 조건문이란



```
if(조건문){ 실행문 }
```

- 일정한 조건이 true로 평가되면 코드 블록이 실행됩니다.
- 조건문은 그 밖의 다른 조건 else if를 가질 수 있습니다.
- 조건문의 else 그 밖의 모든 조건을 가질 수 있습니다.
- if(...)의 조건문은 표현식을 평가하고 그 결과값을 boolean값으로 반환합니다.

조건문이란?

☑ if



if(조건문) { 실행문 }

```
let year=parseInt(prompt('지금은 몇년도 일까요?',''))
if (year == 2026 ) {
  console.log('정답입니다. ');
}
```

- 사용자에게 입력을 받아 조건이 true이면 코드 블록을 실행합니다.

조건문이란?

☑ if else



if(조건문) { 실행문 } else {실행문}

```
let bool = confirm('오늘 비가 오나요?');

if (1) {
  console.log('우산을 챙겨 나가요')
} else {
  console.log('가볍게 나가요')
}
```

- 조건문이 거짓일때 코드 블록을 진행합니다.

조건문이란?

☑ if ~else if



if(조건문) { 실행문 } else if {실행문}

```
let bool = confirm('오늘 비가 오나요?');

if (1) {
  console.log('우산을 챙겨 나가요')
} else {
  console.log('가볍게 나가요')
}
```

- 첫 번째 조건문이 거짓이고, 다음 조건이 참일 때 코드 블록을 진행합니다.

☑ 생각해보기



왜 if(1)은 **true**일까?

조건문 자리에 boolean이 아닌 값을 넣어도 JavaScript는 알아서 true나 false로 해석합니다. 이때 false로 취급되는 값들을 **Falsy**, 나머지를 모두 **Truthy**라고 부릅니다.

☑ 생각해보기



어떤 값이 Falsy일까?

false

0

''

null

undefined

NaN

이 6개를 제외한 **모든 값은 Truthy**

👉 생각해보기



```
if (1) // true, 0이 아닌 숫자는 모두 true
if (-1) // true, 음수도 마찬가지
if (0) // false
if ('hello') // true
if (' ') // true
// 공백 한 칸도 문자가 있으니 true
if ('') // false
if ([]) // true, 빈 배열도 true
if ({}) // true, 빈 객체도 true
if (null) // false
if (undefined) // false
```

빈 배열[]과
빈 객체{}는
Truthy

👉 생각해보기



```
// 값이 들어있는지 짧게 확인 — 자주 쓰입니다
if (username) {
  // username이 빈 문자열, null, undefined가 아닐 때만 실행
}
```

```
// 배열이 비어있는지 확인할 때는 length를 직접 봅니다
if (items.length === 0) {
  // [] 자체는 truthy라서 if(items)로는 못 잡습니다
}
```

```
// React 조건부 렌더링에서도 이 개념이 그대로 적용됩니다
{count && <Badge count={count} />}
// ▲ count가 0이면 '0'이 화면에 그대로 출력됩니다
// → count > 0 && ... 처럼 명시적으로 쓰는 게 안전합니다
```


③ 3항 연산자



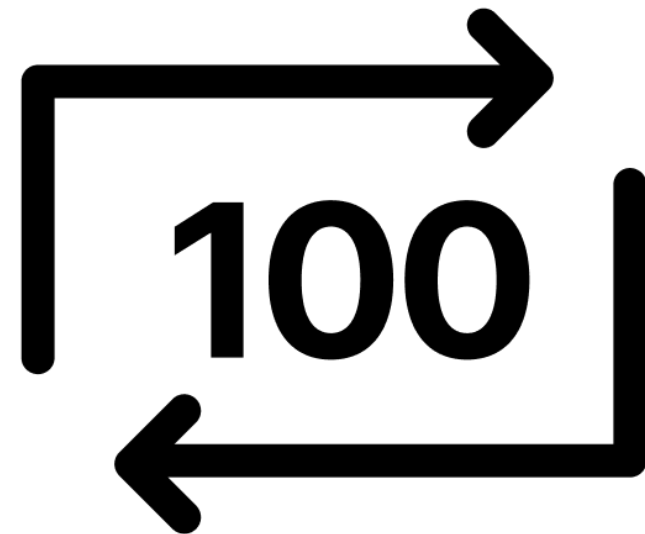
(조건문) ? true 실행문 : false 실행문

```
let age = 26;  
let beverage = (age >= 21) ? "Beer" : "Juice";  
console.log(beverage); // "Beer"
```

- 조건 (삼항) 연산자는 JavaScript에서 세 개의 피연산자를 받는 유일한 연산자입니다.
- 앞에서부터 조건문, 물음표(?), 조건문이 참(truthy)일 경우 실행할 표현식, 콜론(:), 조건문이 거짓(falsy)일 경우 실행할 표현식이 배치됩니다.
- 해당 연산자는 if...else문의 대체재로 빈번히 사용됩니다.

목록

☑ 반복문의 정의



똑같은 작업을 100번 해!

루프는 어떤 것을 반복적으로 시행할 때 빠르고 간편한 방법을 제공합니다.

지금부터 JavaScript 에서 사용이 가능한 서로 다른 여러가지 반복문을 소개합니다.

☑ for문



for (초기식 ; 조건식 ; 증감식){ ...반복 수행 코드... }

```
let num = '';  
  
for(let i=0;i<10;i++){  
    num = num + i + ',';  
}  
console.log(num);
```

- for문은 초기식, 조건식, 증감식을 모두 포함한 반복문입니다.

☑ for / in 문



```
const obj = {  
  name : "elice",  
  age : 20,  
  hobby:"coding"  
};  
for (const key in obj){  
  console.log(key + ":" + obj[key])  
}
```

```
for ( 변수 in 객체 ){  
    ...객체의 프로퍼티 개수 만큼 실행...  
}
```

(주의)해당 객체가 상속받는 프로토타입 체인상의 모든 프로퍼티 키를 열거한다.

(단, [[Enumerable]] 값이 false인 프로퍼티는 제외)

✔ for / of 문

for (변수 of 객체){ ...객체의 프로퍼티 개수 만큼 실행... }

```
const arr = [10, 20, 30];  
for (const item of arr){  
    console.log(item); // 10, 20, 30 출력  
}
```

이터러블 순회 전용

이터러블에는 String, Array, Map, Set, DOM컬렉션(HTMLCollection, NodeList) 등이 있다.

📖 용어 해설

iterable 객체

반복 가능한(iterable, 이터러블) 객체는 배열을 일반화한 객체입니다. 이터러블이라는 개념을 사용하면 어떤 객체에든 for..of 반복문을 적용할 수 있습니다. 배열은 대표적인 이터러블입니다. 배열 외에도 다수의 내장 객체가 반복 가능합니다. 문자열 역시 이터러블의 예입니다.

☑️ forEach()



객체.forEach(function(변수) { ...객체의 요소 개수만큼 실행... })

```
const arr = ['elice', 'baby', 'mike'];

arr.forEach((value, index) => {
  document.write(`${index}:${value}`);
});
```

배열 순회 전용 메소드

콜백함수와 매개변수로 value에 요소값, index에 인덱스, array에 원본 배열이 들어옵니다.

Object.keys() / Object.values()

✔ Object.keys() / Object.values()



```
const arr = {  
  name: 'elice',  
  age: 20  
}  
Object.keys(arr); //['name', 'age']  
Object.values(arr); //['elice', 20]
```

객체 순회 전용 메소드

Object.keys() : 객체 프로퍼티의 key값을 배열로 반환

Object.values() : 객체 프로퍼티의 value값을 배열로 반환